

Operating Systems — Homework 1

Exercise 1.1 – 1.4

Ex. 1.1 (5 pts) — Magic number of a binary executable

Role. A magic number is a short, fixed byte pattern that a file carries so that its *format* can be recognised from its contents rather than from its name. When the kernel services an `execve()`, it does not trust the file extension: it reads the first handful of bytes and matches them against the formats it knows, and that match is what selects the right loader. If nothing matches, the call is simply refused with `ENOEXEC` instead of the machine jumping into arbitrary bytes. It therefore does two jobs at once: format dispatch and a cheap integrity/sanity check.

Where it is stored. In the very first bytes of the file, at offset 0, as the opening field of the file header, which is exactly what makes the check cheap, since the kernel already has to read that first block anyway. e.g.: an ELF binary starts with `0x7F 'E' 'L' 'F'` (the first four bytes of `e_ident` in the ELF header); a shell or Python script starts with `#! (0x23 0x21)` followed by the interpreter path; a Windows PE image starts with `'M' 'Z'`; a Java class file starts with `0xCAFEBAFE`.

Ex. 1.2 (5 pts) — Segments of an address space

A process address space is built from the following regions, conventionally laid out from low addresses to high:

- **Text (code) segment:** the machine instructions of the program. Mapped read-only and executable, so it can be shared by every process running the same binary and cannot be scribbled over by a stray pointer. Fixed size.
- **Initialized data segment (.data):** global and static variables whose initial value is not zero. Their values are stored in the executable file and copied in by the loader. Read/write, fixed size.
- **Uninitialized data segment (BSS):** global and static variables that start out zero. Only their *size* is recorded in the file, not their contents; the loader zero-fills the region, so the BSS costs nothing on disk. Read/write, fixed size.
- **Heap:** memory obtained at run time by `malloc/new`. It sits above the BSS and grows upward, toward higher addresses, as the allocator extends it (`brk/sbrk`, or `mmap` for large requests).
- **Memory-mapped region:** shared libraries, files mapped with `mmap`, and large allocations. It occupies the middle ground between heap and stack, and is how dynamically linked code is brought into the space.
- **Stack:** one per thread; holds activation records: parameters, local variables, saved registers and return addresses. It is placed at the top of the space and grows downward, toward lower addresses, one frame per call.

Ex. 1.3 (5 pts) — Von Neumann machines and machines today

A von Neumann machine is the *stored-program* design: a CPU (control unit plus ALU and registers), a single main memory that holds instructions *and* data indistinguishably, and I/O, all joined by a shared bus. Its defining idea is that the program is itself just data sitting in memory, so the machine is general purpose and it operates by repeating the "fetch->decode->execute" cycle, with a program counter naming the next instruction. The price of the single shared memory and bus is that instruction fetch and data access compete for the same path, the well-known *von Neumann bottleneck*.

Today's machines are still von Neumann at the level the programmer observe, like x86, ARM etc, all present one flat address space holding both code and data, a program counter, and sequential instruction semantics.

Ex. 1.4 (5 pts) — “Segmentation fault (core dumped)”

What the message means. The program touched memory it was not entitled to touch. The MMU checks every address as it translates it; if the address is unmapped, or mapped with the wrong permission (writing a read-only page, executing a non-executable one), it raises a hardware fault. The kernel finds no legitimate reason for it, converts it into `SIGSEGV`, and delivers it to the process, whose default action is to die and write a core dump. The line you see is the shell afterwards reporting how its child died.

Possible reasons.

- Dereferencing a null pointer, typically an unchecked return from `malloc` or `fopen`.
- Dereferencing an uninitialized or dangling pointer, including use-after-free.
- Running past the end of an array far enough to leave the mapping.
- Writing through a `char *` that points at a string literal, which lives on a read-only page.
- Unbounded recursion overflowing the stack.

What “segmentation” means here. The word is historical. On segmented machines the address space was divided into segments, each with a base, a limit and access rights, and the hardware trapped any reference falling outside them: a *segmentation violation*. Mainstream systems are paged rather than segmented now, but the name stuck and still means an access outside a valid, permitted region.

What “core” means here. “Core” is magnetic-core memory, used to be the ferrite-ring main memory of machines of the 1950s. Then it became a synonym for main memory and outlived the technology. A core dump is an image of the process's memory and register state at the instant it faulted, written to a file so a debugger can reconstruct where the program died.

Exercise 1.5 – 1.8

Ex. 1.5 (5 pts) — What a compiler may assume about loading

Why the compiler cannot know. A program is compiled once but run many times, on machines the compiler will never see. Where the binary ends up depends entirely on conditions that exist only at the moment of launch: how much physical memory the machine has, which other processes

are already resident, and therefore which free hole the loader happens to pick. Nothing about that is available while the source is being translated. Modern systems push the answer even further away from compile time: address-space layout randomisation deliberately relocates the image on every run, shared libraries are mapped wherever there is room, and demand paging means a page is not given a physical frame until it is first touched. Two runs of the same binary on the same machine can therefore sit at different addresses, so no fixed address could have been correct anyway.

What it assumes instead. The compiler sidesteps the problem by generating code into a *logical* (virtual) address space that it assumes **begins at address 0** and is **contiguous**. Every internal reference (branch targets, function entry points, addresses of globals) is expressed as an offset from that assumed origin, so the whole image can later be shifted anywhere as a unit. The actual binding of logical to physical addresses is deferred to somebody better informed: either a relocating loader that patches the addresses when it places the image, or hardware (a relocation/base register and the MMU) that adds the offset on every reference at run time. This is exactly what makes *relocatable* and *position-independent* code possible. The three binding times are worth naming: *compile time* (absolute code, only usable if the load address is known in advance), *load time* (relocatable code, fixed up once when loaded), and *execution time* (bindings resolved on each reference, which is what lets a process be moved while running — and what essentially all general-purpose operating systems use today).

Ex. 1.6 (5 pts) — From double-click to window on screen

The two cases differ only in how the command line is assembled. A double-click makes the desktop environment look up the icon's file association to recover an executable path and arguments; a typed command makes the shell tokenise the line, expand variables and globs, and resolve the name against `PATH`. Once a path and an argument vector exist, the rest is identical.

The shell or launcher calls `fork()`, giving a child whose address space is shared copy-on-write, and the child calls `exec()` on the resolved path. The kernel checks the execute permission, opens the file, reads its header and validates the magic number, which identifies the format and selects the loader; a bad magic number fails the `exec` rather than running garbage. The child's old address space is torn down and a new one built: text, initialised data, zero-filled BSS, heap and stack, with `argv` and `envp` copied onto the new stack. The segments are memory-mapped rather than read in, so almost no I/O happens yet. If the binary is dynamically linked, the interpreter named in its header (`ld.so`) is mapped and runs first, mapping the shared libraries and resolving symbols. The kernel then updates the process control block (page tables, inherited file descriptors, reset signal handlers) and puts the process on the ready queue. When the scheduler dispatches it, execution begins at the entry point and pages are faulted in as they are touched. Startup code initialises the language runtime and calls `main()`, which connects to the window system, creates a window and draws into it; the compositor puts those pixels on the screen.

Ex. 1.7 (5 pts) — Virtual to physical under contiguous allocation

Under contiguous allocation each process occupies one unbroken block of physical memory, which makes translation extremely cheap: the entire mapping is described by two registers that the kernel loads on a context switch, the **relocation (base) register** holding the block's starting physical address and the **limit register** holding its length. The MMU applies them in a fixed order: *check*

first, then add:

$$\text{physical} = \begin{cases} \text{base} + \text{virtual}, & \text{if virtual} < \text{limit}, \\ \text{addressing trap to the OS}, & \text{if virtual} \geq \text{limit}. \end{cases} \quad (1)$$

For example, with base = 14000 and limit = 3000:

$$\begin{aligned} \text{virtual } 346 : 346 < 3000 &\Rightarrow \text{physical} = 14000 + 346 = 14346, \\ \text{virtual } 3200 : 3200 \geq 3000 &\Rightarrow \text{addressing trap (no access)}. \end{aligned}$$

The process only ever names logical addresses in $[0, \text{limit})$ and never sees a physical address, so it is structurally incapable of referring to memory outside its own block — that is where the protection comes from. Loading the base and limit registers is a privileged instruction available only to the kernel, which is what stops a process from simply widening its own bounds.

Ex. 1.8 (5 pts) — The round-robin scheduler

Round robin is first-come-first-served made preemptive by a clock. The ready queue is treated as a circular FIFO and each process, on reaching the front, is given the CPU for at most one fixed **time quantum** q (typically tens of milliseconds); if it is still running when the timer interrupt fires, the OS preempts it, saves its context and moves it to the tail of the queue, while a process that blocks or finishes early simply gives the CPU up sooner. With n runnable processes, every one of them therefore receives roughly $1/n$ of the CPU in slices no longer than q and waits no more than about $(n - 1)q$ for its next turn.

The choice of q is the whole design decision: too large and no process is ever actually preempted, so the algorithm degenerates into plain FCFS, while too small and the system spends a growing fraction of its time context-switching instead of doing work — q must stay large relative to the context-switch cost. Its virtues are fairness and a bounded, predictable response time with no possibility of starvation, which is why it suits interactive and time-sharing systems. Its vices are a comparatively poor average turnaround time (worse than SJF) and the fact that it is deliberately blind: it takes no account of priority, deadlines, or how much work a job actually has left.

Ex. 1.9 — Three Papers from “OS History and Architecture”

Paper 1-10 — Dijkstra, “The Structure of the “*THE*”-Multiprogramming System”

Summary. Dijkstra reports a multiprogramming system built at Eindhoven for the Electrologica EL X8 by six people of on average half-time availability. The goals are modest: shorter turnaround for short jobs, economical use of peripherals, automatic control of backing store; it is explicitly not a multiaccess system.

Two ideas carry the design. The first decouples information from storage: segments are named independently of pages, so a segment evicted to the drum need not return to the page it came from and the free page with minimum latency is simply selected. The second is the level hierarchy, a society of sequential processes at undefined speed ratios coordinated only by **P** and **V** on semaphores, stacked strictly from processor allocation at level 0 up to user programs, each level implementing an abstraction that makes the one below it invisible.

The claimed payoff is verification rather than speed. Because a process can only generate tasks for processes at lower levels, “circularity is excluded”, and harmonious cooperation is proved in roughly three stages, ending in the absence of what Dijkstra calls the “Deadly Embrace”. Testing found only trivial coding errors, one per 500 instructions, each located within ten minutes.

Critical judgment. The case for layering is purely structural: the paper contains no performance data at all, so we are never told what the hierarchy cost in memory, in context switches, or in latency. The most revealing sentence is the concession that testing was not yet complete “but the resulting system is guaranteed to be flawless”. That offers the rigour of a design in place of evidence about an artifact, and it sits badly beside his own report a paragraph earlier: the proof covers the synchronisation skeleton, while the errors he found were coding errors, exactly the class it cannot reach.

The ordering also shows a crack the paper itself records. Keeping the hierarchy acyclic required a segment fetched from the drum to be pinned in core until the requesting process has accessed it, since otherwise “finite tasks could be forced to generate an infinite number of tasks for the segment controller”. Acyclicity bounds which processes may be asked to do work, but not how often, so termination had to be rescued by a rule living outside the hierarchy. The strict total order did not survive: in later systems the pager needs the disk driver, which needs memory, which needs the pager.

Scope does quiet work throughout. Six people, one configuration, no multiaccess and no user-written machine code is a setting where exhaustive testing is plausible in a way it is not elsewhere. Dijkstra anticipates the objection and answers that “the larger the project, the more essential the structuring!” That is asserted rather than argued. What survived is the thesis beneath it: choose a structure because it makes correctness provable.

Paper 1-2 — Ritchie and Thompson, “The UNIX Time-Sharing System”

Summary. This describes the third version of UNIX, on the PDP-11/40 and /45. The scale is part of the argument: 144K bytes of core of which UNIX occupies 42K, hardware costing as little as \$40,000, less than two man years on the main system software, and about 40 installations since February 1971. It had been rewritten in C in 1973 at about one third greater size, accepted for the gain in ease of modification.

The design is a small set of uniform abstractions. Ordinary files are unstructured strings of bytes, so structure belongs to the programs that use a file and not to the system; every device appears as a special file, under one name syntax and one protection mechanism; all links to a file have equal status, a directory entry holding only a name and an i-node pointer; and processes are built from `fork`, `execute`, `wait` and `pipe`. The Shell is not part of the kernel but an ordinary swappable user program, and because the Shell rather than the command interprets `<`, `>` and `|`, no command contains any code for redirection. Section 9 reports 72 users, 14 maximum simultaneous, and 1800 commands a day at about 98 percent uptime.

Critical judgment. The entire performance argument is section 4.1: one assembly of a 7621-line program in 35.9 seconds, divided 63.5 percent assembler execution, 16.5 percent system overhead and 20 percent disk wait. The authors then decline to interpret the figures or compare them with any other system, saying only that they are generally satisfied. A design paper is entitled to that stance, but the consequence is that every efficiency claim in the paper rests on assertion.

The treatment of security is thin, and the mechanisms introduced most casually became the classic

problems. Protection is a user ID plus seven bits, with no group in this version, plus set-user-ID and a super-user exempt from all checking. Set-user-ID is presented purely as an elegance, available to any user on his own files without administrative intervention. The paper never asks what happens when such a program can be induced to act on its invoker's behalf, and decades of privilege-escalation bugs live in that gap.

“Everything is a file” is also less total than the slogan it became, and the authors say so where it is inconvenient: pipes are conceded to be “not a completely general mechanism since the pipe must be set up by a common ancestor”, which is why named pipes and then sockets had to be added later. Section 9 also describes a research laboratory rather than a production load, chess alone taking 5.3 percent of command CPU time, so the reliability figures cannot support conclusions about UNIX under commercial pressure.

The Perspective section is candid about circumstance rather than method: the authors were never faced with the need “to satisfy someone else's requirements”. That is honest, and it is also the part of the story that cannot be copied. What the paper does establish is its own opening claim, that a powerful interactive system “need not be expensive either in equipment or in human effort”.

Paper 1-3 — Engler, Kaashoek and O'Toole, “Exokernel”

Summary. The target is the fixed abstraction itself: any abstraction the kernel imposes embodies a trade-off, and whichever way it is resolved some class of application pays. The authors cite Cao et al., who report that application-level control over file caching alone can reduce running time by 45 percent. Their response is to separate protection from management: the kernel securely multiplexes the raw hardware and nothing more, while untrusted *library operating systems*, linked into the application's own address space, implement processes, virtual memory and IPC. Changing the operating system becomes relinking.

Three mechanisms make this safe: *secure bindings*, which let the kernel perform the expensive check once and enforce it cheaply thereafter; *visible revocation*, which makes the library a participant rather than a victim; and an *abort protocol* that breaks bindings by force when a library will not cooperate. The prototype, Aegis with ExOS above it, is compared against Ultrix 4.2 on MIPS DECstations. Exception dispatch takes $1.5\ \mu\text{s}$ against Ultrix's 130 and five times the best reported implementation, protected control transfer is almost seven times the best reported, the dynamic packet filter classifies TCP/IP headers in $1.5\ \mu\text{s}$ against 35 for MPF, and an application-level stride scheduler takes under 100 lines of code.

Critical judgment. The diagnosis is excellent and the evaluation cannot carry it. Nearly every number is a microbenchmark of a single primitive, averaged over many repetitions, which as the authors state themselves means the measurements “do not consider cold start misses in the cache or TLB, and therefore represent a best case”. A system whose entire premise is that real applications are held back by the kernel is evaluated without a real application; the one whole-program measurement, a 150×150 matrix multiplication, is dominated by the compiler.

The comparison is also structurally unfair in a way the paper concedes in one sentence and then walks past: ExOS does “not offer the same level of functionality as Ultrix”, and the authors “do not expect these additions to cause large increases in our timing measurements”. That expectation is exactly the claim that needed evidence, because the absent functionality, swapping and a file system still under development, is where a general-purpose system spends its money. Read alongside the admission that the prototype “has no real users”, the ten-to-100 $\times$ speedups are a weaker result

than the number suggests, since the baseline is doing strictly more work.

The security argument is asserted more than demonstrated: no adversary model is developed and no attack attempted. The requirement that a packet filter must not “lie” and accept packets destined for another application is answered with a trusted installer, or, on “a system that assumes no malicious processes”, by statically checking the filter language, a premise that assumes away the threat the mechanism was meant to address. The paper prefers the second route, since avoiding a central authority “increases extensibility”, which makes the weaker premise the load-bearing one. The deeper unresolved tension is portability, since exposing physical names to untrusted software makes the interface hardware-specific by construction. Their treatment of VM/370 shows it: they grant that it “exports the ideal exokernel interface” and separate themselves from it on cost, that virtualising the base machine “can be expensive” and “often requires additional hardware support”. That support duly arrived, which weakens the cost objection; what survives is their second one, that hiding the real machine leaves applications managing virtual resources counterproductively. The critique of fixed abstractions was right and has outlived the architecture built on it.

References

- [1-10] E. W. Dijkstra. “The Structure of the “*THE*”-Multiprogramming System.” *Communications of the ACM*, vol. 11, no. 5, May 1968, pp. 341–346. (Presented at the ACM Symposium on Operating System Principles, Gatlinburg, Tennessee, October 1967; also circulated as EWD196.)
- [1-2] D. M. Ritchie and K. Thompson. “The UNIX Time-Sharing System.” *Communications of the ACM*, vol. 17, no. 7, July 1974, pp. 365–375. (Revised version of a paper presented at the Fourth ACM Symposium on Operating Systems Principles, Yorktown Heights, New York, October 1973.)
- [1-3] D. R. Engler, M. F. Kaashoek and J. O’Toole Jr. “Exokernel: An Operating System Architecture for Application-Level Resource Management.” *Proceedings of the Fifteenth ACM Symposium on Operating Systems Principles (SOSP ’95)*, 1995, pp. 251–266.

Ex. 1.0 — Three Papers on OS History and Architecture

Paper 1 — Corbató and Vyssotsky, “Introduction and Overview of the Multics System” (1965)

Summary. This paper opens the six-paper Multics session at the 1965 Fall Joint Computer Conference. Its problem is access: under batch processing, user contact with large machines had “retrogressed”, isolating the programmer from cause and effect. The goal is a *computer utility*, running “continuously and reliably 7 days a week, 24 hours a day in a way similar to telephone or power systems”.

The mechanisms are co-designed with the GE 645. Addressing is two-dimensional: programs are written as segments that grow or shrink during execution, up to a quarter million per user of a quarter million words each, paged with either 64- or 1,024-word pages, and cross-segment references are bound by dynamic linking at first reference. Pure shareable procedures are the normal mode, so there is “no clear-cut demarcation between user programs and system programs”; protection comes

from descriptor bits acting as hardware “fire-walls”; processors form an anonymous pool in which “the supervisor does not have a special processor”; and PL/I is adopted for machine independence. There is no evaluation, since the system did not yet exist, and the one quantitative claim, “a few hundred” simultaneous users by “simple scaling of processor and memory speed”, is immediately called “hazardous”.

Critical judgment. This is a prospectus, and nothing in it is falsifiable by its own evidence. The deeper omission is not measurement but cost. The paper proposes to meet “almost all of the present and near-future requirements” of a large installation and never asks what that generality would cost to build, which is precisely where the project failed: development ran years late, Bell Labs withdrew in 1969, and UNIX was written shortly afterwards in reaction to the scale. Its most accurate sentences are its own hedges, that the plans are “not unattainable” but that it would be “presumptuous” to expect the initial system to meet all of them.

History has split the substance cleanly. Vindicated: paged virtual memory, device-independent I/O, a symbolically named hierarchical file system with per-file access control and backup, symmetric multiprocessing with no master, and writing an operating system in a high-level language. Refuted: user-visible segmentation lost to the flat paged address space, and x86-64 all but abolished it; dynamic linking survived, but as lazy binding over a flat space rather than as segment binding.

The computer utility is the interesting case. The economic argument, that “bulk economies” and “floor space, management efficiency and operating personnel” favour “centralizing computer facilities in a single large installation”, was falsified for some thirty years by the minicomputer and the personal computer, then vindicated by cloud computing. The paper was right about the destination and wrong about the reason, since what made centralisation win was statistical multiplexing at datacentre scale.

Paper 2 — Accetta et al., “Mach: A New Kernel Foundation for UNIX Development” (1986)

Summary. Mach’s target is the accretion of mechanism in UNIX. A system that began with a small uniform set of operations on file descriptors had grown “a staggering number” of overlapping facilities, from streams and sockets to “a mind-boggling array of `ioctl` operations”, while the process abstraction remained too heavyweight to express parallelism.

Mach replaces the base with four abstractions. A *task* owns a paged address space and its port rights; a *thread* is the unit of CPU utilisation, so a UNIX process is a task with one thread; a *port* is a kernel-protected message queue acting as a capability, with any number of senders but one receiver; and a *message* is typed data that may carry port rights and may be as large as an entire address space. Every operation on any other object is a message to the port representing it, so that “the actual system running on any particular machine is a function of its servers rather than its kernel”.

The virtual memory system offers per-page inheritance and a current-versus-maximum protection pair in which the maximum “may never be raised, it may only be lowered”, with copy-on-write implementing both `fork` and bulk message transfer. Most consequentially, page faults may be serviced outside the kernel by user-level pagers, so a memory-mapped file is simply memory whose pager is the file system. As of April 1986 Mach was binary compatible with 4.3BSD and ran on most VAX machines, a four-processor VAX 11/784 and the IBM RT/PC.

Critical judgment. The gap between the ambition of the design and the weight of the evidence is

very wide, and the paper is candid that “extensive performance comparisons with other systems have not yet been done”. Its entire quantitative evaluation is one number: on a MicroVAX II, touching newly allocated memory costs “less than 0.7 milliseconds per 1024 bytes of data (versus approximately 1.2 milliseconds for 4.3BSD)”. That microbenchmark measures zero-fill fault cost, exactly what the new VM system was built to improve, and `fork` is called “substantially faster” with no figure attached at all.

Nothing measures IPC latency, although ports and messages are the centrepiece of the design; nothing measures multiprocessor scaling, although the abstract’s opening sentence announces a multiprocessor kernel; threads were not yet implemented and were “expected by Summer 1986”; and the caption of Figure 6 concedes that as of April 1986 the box labelled UNIX compatibility “still executes in kernel state”. The central claim, that operating system services can be moved out to user-level servers at acceptable cost, is precisely the claim the paper does not test.

History then split the verdict. The mechanisms won comprehensively: separating the address space from the schedulable entity is now universal, memory-mapped files and external pagers are standard, the machine-dependent split survives as the `pmap` layer in the BSDs, and macOS and iOS run on a Mach derivative to this day. The performance thesis lost: Mach 3.0’s user-space UNIX server proved slow, Chen and Bershad traced the degradation to memory-system behaviour rather than message path length, and Liedtke’s L4 work argued the cost was an artefact of implementation rather than of microkernels.

Paper 3 — Baumann et al., “The Multikernel: A New OS Architecture for Scalable Multicore Systems” (2009)

Summary. The premise is that hardware now diversifies faster than system software can be retuned for it, so an OS built as a shared-memory kernel with lock-protected data structures faces an unbounded engineering bill; the authors evidence this with the removal of the Windows 7 dispatcher lock, which “touched 6000 lines of code in 58 files” and was described as heroic. Arguing that the machine has itself become a network, they propose three principles: make all inter-core communication explicit, make OS structure hardware-neutral, and treat state as replicated rather than shared.

Their prototype, Barrelfish, splits the per-core OS instance into a privileged *CPU driver* (7135 lines of C for x86-64) and a schedulable user-space *monitor* that holds the replicas and runs agreement protocols over URPC, a message transport tuned to the cache-coherence protocol. The motivating measurement is stark: a single core updates shared state in under 30 cycles, but with 16 cores on the same data each update costs “almost 12,000 extra cycles”, and a single dedicated server “can perform twice as many updates per second as all 16 shared-memory threads combined”. The evaluation then shows NUMA-aware multicast shutdown holding near 3.1k cycles across 32 cores where broadcast and unicast climb to roughly 13.8k and 12.7k; unmapping at 32 cores costing about 17.5k cycles against 29.5k for Linux and 46k for Windows; and a static web server at 18,697 requests per second against 8924 for `lighttpd` on Linux.

Critical judgment. This is much the strongest evaluation of the three and an unusually honest one. It states outright that “it would be wrong to draw any quantitative conclusions from our large-scale benchmarks”, concedes that the capability system was “a mistake... unnecessarily complex, and no more efficient than scalable perprocessor memory managers used in conventional OSes like Windows and Linux”, calls its network stack “very much a placeholder”, and admits every test platform was

homogeneous x86-64, so the heterogeneity motivating the whole argument was never exercised.

That honesty makes the weak points easier to locate. The web-server win pits a purpose-built user-space lwIP stack against general-purpose lighttpd on Linux with offload enabled that Barrelfish’s driver did not support, so what is measured is specialisation rather than architecture. The scalability case rests on global-coordination microbenchmarks, and the gain there comes largely from replacing serial IPIs with a topology-aware multicast tree, an algorithmic change a shared-memory kernel could equally adopt, so the experiment does not isolate shared-nothing structure as the cause.

Reading the two figures against each other is more revealing. The raw messaging baseline costs about 3.1k cycles at 32 cores while the complete unmap costs about 17.5k, so more than four-fifths of the operation is Barrelfish’s own monitor LRPC, marshaling and unoptimised thread dispatch. The unmap curve makes the same point at the other end: Barrelfish does not start cheaper, costing about 10.3k cycles at two cores against roughly 5.2k for Linux and 5.8k for Windows, and what it buys is slope, growing about $1.7\times$ to 32 cores where Linux grows $5.7\times$ and Windows $7.9\times$. The multikernel did not reduce the constant cost of the operation; it roughly doubled it and bought a much flatter curve. That is the trade the authors concede, that routing invocations through monitor RPC adds “a constant overhead on current hardware of several thousand cycles” which is “constant as the number of cores increases”, a bargain that pays only once the machine is wide enough and whose break-even against Linux does not arrive until around twelve or thirteen cores. The messaging table reinforces the point, showing URPC at 450 cycles of latency against 424 for L4’s decade-older IPC, the real gains being throughput and cache footprint rather than latency.

References

- [1] F. J. Corbató and V. A. Vyssotsky. “Introduction and Overview of the Multics System.” In *Proceedings of the AFIPS Fall Joint Computer Conference (FJCC)*, vol. 27, part 1, pp. 185–196, 1965.
- [2] M. Accetta, R. Baron, W. Bolosky, D. Golub, R. Rashid, A. Tevanian and M. Young. “Mach: A New Kernel Foundation for UNIX Development.” In *Proceedings of the USENIX Summer Conference*, pp. 93–112, 1986.
- [3] A. Baumann, P. Barham, P.-E. Dagand, T. Harris, R. Isaacs, S. Peter, T. Roscoe, A. Schüpbach and A. Singhanian. “The Multikernel: A New OS Architecture for Scalable Multicore Systems.” In *Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP)*, pp. 29–44, 2009.